

Représentation des données : le binaire et ses usages

Toute machine informatique manipule des données sous forme de **bits** (0 ou 1). Un bit est l'unité minimale d'information. *Tout* peut être stocké et traité comme une suite de 0 et de 1 (nombres, texte, images, son...).

Un groupe de 8 bits forme un **octet** (*byte*). Les tailles courantes en mémoire sont 8, 16, 32 ou 64 bits.

Remarque : pourquoi *byte* ? pourquoi 8 bits ? ▼

Le mot **byte** désigne en anglais la plus petite unité adressable de mémoire. C'est un jeu de mots volontaire avec *bite* (bouchée), pour éviter toute confusion avec *bit*. Un *byte* est une « bouchée de données », par analogie avec *bit* (abréviation de **binary digit**) qui signifie aussi « petit morceau » en anglais. La hiérarchie est donc : un *bit* est un fragment, un *byte* est une bouchée. Mais la *taille* de la bouchée a fluctué au fil du temps !

Dans les années 1960-70, les machines utilisaient des mots de 6, 7, ou 9 bits selon les constructeurs. C'est l'architecture de l'**IBM System/360** (1964) qui a popularisé le *byte* de 8 bits, appelé octet en français. Cette taille est en effet une bonne unité de base : assez grande pour coder un caractère ASCII (7 bits + 1 bit de parité), et puissance de 2 (pratique pour les circuits numériques).

Les processeurs modernes traitent les données par mots de 32 ou 64 bits, mais **l'octet reste l'unité d'adressage de la mémoire** : chaque adresse mémoire pointe vers un octet, pas vers un mot entier. C'est pourquoi on dit qu'un `int` sur 64 bits occupe 8 **adresses** mémoire consécutives.

En Python :

```
import sys
sys.getsizeof(42)      # 28 octets – surcoût lié à l'objet Python sous-jacent
sys.getsizeof(2**100)  # Plus grand : entier de taille arbitraire
```

1. Bases de numération

1.1. Principe

En base b , on dispose de b chiffres (de 0 à $b - 1$). La valeur d'un nombre s'obtient en multipliant chaque chiffre par la puissance de b correspondant à sa position :

$$(d_n d_{n-1} \dots d_1 d_0)_b = \sum_{k=0}^n d_k \times b^k.$$

Analogie

On compte par paquets de b , avec b symboles différents : 0, 1, 2, ... $b - 1$. Arrivé à $b - 1$, lorsqu'on ajoute une unité, on remet un 0 et on ajoute un 1 devant. On a alors 1 paquet de b et 0 unités.

On compte habituellement en décimal (base 10), avec les 10 symboles habituels : on fait des paquets de 10 (les dizaines). 10 signifie « 1 dizaine et 0 unité », 207 signifie « 2 dizaines de dizaine (aussi appelées *centaines*), 0 dizaine et 7 unités ».

En base 2, on n'emploie que deux symboles : on compte 0, puis 1, et on a déjà épuisé les symboles disponibles ! On fait donc un paquet de 2 (une « deuzaine »), et on inscrit 10_b pour signifier « 1 paquet de 2 unités et 0 unité seule ». Le nombre suivant, 11_b , signifie « 1 paquet de 2 et 1 paquet de 1 ». L'épuisement des symboles se pose à nouveau pour le nombre d'après : on écrit donc 100_b , pour « 1 paquet de 2 deuzaines » (une « quatraine »), 0 « deuzaine » et 0 unités. Et ainsi de suite.

On peut compter en n'importe quelle base (ex. : en base 4 !).

Bases utilisées en informatique :

Base	Nom	Chiffres	Préfixe Python
2	Binaire	0, 1	0b
10	Décimale	0–9	Aucun
16	Hexadécimale	0–9, A–F	0x

La base 8 (l'octal) a aussi été utilisée en informatique.

Exemple de lecture :

- $(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$
- $(2A)_{16} = 2 \times 16^1 + 10 \times 16^0 = 32 + 10 = 42$

Remarque : poids fort, poids faible ▼

Dans l'écriture 11010_b , le bit le plus à gauche (1 ici) est qualifié de « bit de poids fort ». C'est le « constituant principal », celui qui « pèse » le plus dans le nombre 11010_b , car il représente $2^4 = 16$.
Le bit le plus à droite (0 ici) est qualifié de « bit de poids faible ». C'est celui qui « pèse » le moins dans le nombre 11010_b , car il compte au mieux pour $2^0 = 1$.

Remarque : notations alternatives ▼

Pour éviter toute confusion liée à des nombres en indice écrits « un peu trop haut », il arrive qu'on écrive 42_d pour indiquer que le nombre 42 s'entend ici en numération décimale. De même, on peut écrire 101_b ou 101_h .
Rappel : 101_d vaut simplement 101 en décimal. Mais $101_b = 5_d$, et $101_h = 257_d$!

1.2. Opérations basiques en binaire

Elles fonctionnent comme leurs équivalents en décimal. Les retenues se propagent de la même façon. Il est essentiel de bien maîtriser les règles de l'addition.

1.2.1. Addition binaire

$0 + 0 = 0$, $0 + 1 = 1 + 0 = 1$; $1 + 1 = 10_b$ (« je pose 0 et je retiens 1 »).

Note : dans un souci d'efficacité, il est pertinent de retenir que $1 + 1 + 1 = 11_b$ (« je pose 1 et je retiens 1 »).

Exemple : une addition en pratique ▼

Effectuons l'addition $111010_b + 11001_b$.

Étape 1	Étape 2	Étape 3	Étape 4
1 1 1 0 1 0 + 1 1 0 0 1 ----- 0 1 1 3 2 1 1 0 + 1 = 1 2 1 + 0 = 1 3 0 + 0 = 0	1 ← 5 1 1 1 0 1 1 + 1 1 0 0 1 ----- 0 0 1 1 4 ► 1 + 1 = 10_b 4 On pose 0 et... 5 ...on retient 1.	1 ← 7 1 1 1 0 1 1 + 1 1 0 0 1 ----- 1 0 0 1 1 6 ► 1 + 1 + 1 = 11_b 6 On pose 1 et... 7 ...on retient 1.	1 1 1 0 1 1 + 1 1 0 0 1 ----- 1 0 1 0 0 1 1 8 ► 1 + 1 = 10_b 8 On écrit 10. C'est fini !

Vérifications

$111010_b = 32 + 16 + 8 + 2 = 58_d$; $11001_b = 16 + 8 + 1 = 25_d$; $58 + 25 = 83_d$ et $1010011_b = 64 + 16 + 2 + 1 = 83$.

✔ C'est correct !

1.2.2. Soustraction binaire

- 1. $0 - 0 = 1 - 1 = 0, 1 - 0 = 1$
- 2. Plus délicate : $0 - 1 = 1$ avec *prélèvement d'une « deuzaine » à gauche !*

 **Dangers !**

- Pour des entiers relatifs, il faut que le nombre que l'on soustrait soit inférieur au nombre duquel on le soustrait.
- Comme pour la soustraction décimale, il importe de se souvenir de la gymnastique des retenues, lorsqu'on emploie la règle 2 :
 - « En haut », le 1 que l'on accole à gauche du 0 forme alors un 10 binaire (2 en décimal).
 - « En bas », pour compenser cet « emprunt » réalisé sur le chiffre plus à gauche de la ligne supérieure, on ajoute 1 au chiffre suivant (plus à gauche) du nombre à soustraire.

Dans ce contexte, c'est bien une addition : il faut donc penser que c'est alors $1 + 1 = 10$ *en binaire*. Les retenues peuvent alors se propager.

 Exemple : une soustraction en pratique ▼

Effectuons la soustraction $1000011_b - 111010_b$.

Étape 1 (facile)	Étape 2 (ça se complique)	Étape 3
$\begin{array}{r} 1\ 0\ 0\ 0\ 0\ 1\ 1 \\ -\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline \end{array}$ 0 0 1 ↑ ↑ ↑ 3 2 1 ❶ $1 - 0 = 1$ ❷ $1 - 1 = 0$ ❸ $0 - 0 = 0$	$\begin{array}{r} 1\ 0\ 0\ 1\ 0\ 0\ 1\ 1 \\ -\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline \end{array}$ 1 0 0 1 4 ► Ligne du dessus : « emprunt » d'une « deuzaine » nécessaire. ⚠ 10 doit être compris comme 10_b. ► Ligne du dessous : on compense l'« emprunt » réalisé au-dessus. ⚠ 11 signifie ici $1 + 1 = 10_b$. ❹ Ligne de résultat : $10_b - 1_b = 1_b$.	$\begin{array}{r} 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\ -\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline \end{array}$ 0 1 0 0 1 5 ► Ligne du dessus : « emprunt » encore nécessaire, $10 \rightarrow 10_b$. ► Ligne du dessous : 11 signifie à nouveau $1 + 1 = 10_b$. ❺ Ligne de résultat : $10_b - 10_b = 0_b$.
Étape 4	Étape 5	Étape 6
$\begin{array}{r} 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1 \\ -\ 1\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline \end{array}$ 0 0 1 0 0 1 6 ► Ligne du dessus : 3^e « emprunt ». $10 \rightarrow 10_b$. ► Ligne du dessous : 11 signifie toujours $1 + 1 = 10_b$. ❻ Ligne de résultat : $10_b - 10_b = 0_b$	$\begin{array}{r} 1\ 0\ 0\ 0\ 0\ 1\ 1 \\ -\ 1\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline \end{array}$ 0 0 0 1 0 0 1 7 ❼ Ligne de résultat : $1 - 1 = 0$.	Vérifications ► $1000011_b = 64 + 2 + 1 = 67_d$ ► $111010_b = 32 + 16 + 8 + 2 = 58_d$ ► $67 - 58 = 9$ ► $1001_b = 8 + 1 = 9$ ✔ C'est correct !

1.2.3. Multiplication binaire

Effectuer manuellement une multiplication revient à effectuer autant d'additions qu'il y a de bits dans le nombre écrit au-dessous, lorsqu'on pose le calcul. Il est *fortement* recommandé de faire les additions 2 par 2, au fur et à mesure, pour éviter une propagation délicate d'un grand nombre de retenues.

$0 \times 0 = 0 \times 1 = 1 \times 0 = 0$ et $1 \times 1 = 1$.

Note : dans un souci d'efficacité, il est pertinent de retenir que $1 + 1 + 1 = 11_b$ (« je pose 1 et je retiens 1 »).

Remarque : cas particulier de la multiplication par 2_d

Multiplier par 2_d revient à multiplier par 10_b . Ainsi, cela revient à **décaler les bits du nombre initial d'un cran vers la gauche, en complétant à droite par un 0**.
 $1011_b \times 10_b = 10110_b$ (équivalent en décimal à $11 \times 2 = 22$).
En Python, cette opération de décalage de bit vers la *gauche* se note `<<` : `11 << 1` donne 22 (décalage d'un bit vers la gauche) ; `11 << 2` donne 44 (décalage de 2 bits vers la gauche, correspondant à deux multiplication successives par 2).

1.2.4. Division binaire

La division posée n'est rien de plus qu'une succession de soustractions.

Remarque : cas particulier de la division par 2_d

C'est l'opération réciproque de la multiplication par 2_d : cela revient donc à **supprimer le bit de *droite* du nombre initial**.
 $11011_b // 10_b = 1101_b$ (équivalent en décimal à $27 // 2 = 13$).
En Python, cette opération de décalage de bit vers la *droite* se note `>>` : `27 >> 1` donne 13 (décalage d'un bit vers la droite) ; `27 >> 3` donne 3 (décalage de 3 bits vers la droite, correspondant à 3 divisions entières successives par 2).

2. Conversions

2.1. Base 10 → Base 2

2.1.1. Méthode 1 : divisions successives par 2

On divise l'entier par 2 en notant les restes, puis on lit les restes de *bas en haut* (mais on écrit bien ces restes de gauche à droite !).

42 ÷ 2 = 21 reste 0

21 ÷ 2 = 10 reste 1

10 ÷ 2 = 5 reste 0

5 ÷ 2 = 2 reste 1

2 ÷ 2 = 1 reste 0

1 ÷ 2 = 0 reste 1

↳ lecture de bas en haut : 101010

Donc $42_{10} = 101010_2$.

2.1.2. Méthode 2 : soustractions de puissances de 2

On identifie la plus grande puissance de 2 inférieure ou égale au nombre à convertir, on la soustrait, et on recommence avec la puissance de 2 inférieure.

42 = 32 + 10 = 32 + 8 + 2 = $2^5 + 2^3 + 2^1$

➡ bits aux positions 5, 3, 1 (comptées à partir de 0 en partant de la droite) : 101010

On retrouve le fait que $42_d = 101010_b$.

2.1.3. En Python

```
bin(42)          # '0b101010' - fonction native
bin(42)[2:]      # '101010'   - sans le préfixe

# Manuellement :
def decimal_vers_binaire(n):
    """Convertit un entier décimal en une chaîne de caractères composée de 0 et de 1"""
    if n == 0:
        return '0'
    resultat = ''
    while n > 0:
        resultat = str(n % 2) + resultat          # n % 2 donne la valeur d'un nouveau bit
        n //= 2                                   # Nouvelle valeur pour n : le résultat de la division entière de n par 2
    return resultat
```

Remarque : plus efficace ▼

Ce code montre une méthode « d'accumulation de caractères » dans la variable `resultat`. Mais celle-ci est constamment écrasée / recrée : c'est peu performant. Une meilleure approche : créer une *liste* Python vide `resultat = []`, accumuler les *bits* dedans avec `resultat.append(str(n % 2))`, puis renvoyer `''.join(reversed(resultat))`.

Attention à bien penser à la conversion de `n % 2` en chaîne de caractères !

2.2. Base 2 → Base 10

Méthode : somme des puissances de 2

Chaque bit à 1 contribue 2^k selon sa position k (comptée de droite à gauche depuis 0).

```
1  0  1  0  1  0
↑  ↑  ↑  ↑  ↑  ↑
25 24 23 22 21 20
= 32 + 0 + 8 + 0 + 2 + 0 = 42
```

En Python :

```
int('101010', 2)    # 42 - fonction native (2e argument = base source)
int('0b101010', 2)  # 42 - fonctionne aussi avec le préfixe

# Manuellement :
def binaire_vers_decimal(s):
    """Convertit une chaîne de caractères composée de 0 et de 1 en le décimal correspondant"""
    resultat = 0
    n = len(s)
    for pos, bit in enumerate(s):
        resultat += int(bit) * 2**(n-1-pos)
    return resultat
```

Remarque : autre méthode ▼

Une autre façon de procéder :

```
def binaire_vers_decimal(lst):
    """Convertit une liste de 0 et de 1 en le décimal correspondant"""
    decimal = 0
    for b in lst:
        decimal = 2*decimal + b
    return decimal
```

Cet algorithme (dû à Horner), parcourt la donnée d'entrée *de gauche à droite*, sans nécessiter de puissances de 2 « compliquées » (façon `2**(n-1-pos)`). Pour mieux percevoir son fonctionnement, il suffit d'afficher à chaque tour de la boucle la représentation binaire du decimal en cours de détermination. La multiplication par 2 du décimal courant revient à *décaler sa représentation binaire d'un cran vers la gauche*. Ainsi, à chaque étape, cet algorithme décale vers la gauche « l'écriture » en cours, et lui adjoint à droite le nouveau bit.

Exemple sur [1, 1, 1, 0, 1, 0] :

```
0b0      # decimal = 0
0b1      # x2 → 0b0      puis +1 → 0b1      et decimal = 1
0b11     # x2 → 0b10     puis +1 → 0b11     et decimal = 3
0b111    # x2 → 0b110    puis +1 → 0b111    et decimal = 7
0b1110   # x2 → 0b1110   puis +0 → 0b1110   et decimal = 14
0b11101  # x2 → 0b11100  puis +1 → 0b11101  et decimal = 29
0b111010 # x2 → 0b111010 puis +0 → 0b111010 et decimal = 58
```

On a bien $111010_b = 58_d$.

2.3. Conversions impliquant l'hexadécimal

```
hex(42)      # '0x2a'  - entier → hexadécimal
int('2a', 16) # 42     - hexadécimal → entier
```

Astuce pour les conversions binaire ↔ hexadécimal : chaque chiffre hexadécimal correspond exactement à un groupe de 4 bits (aussi appelé *quartet* en français). La conversion est facile et quasiment sans calcul ! Par exemple : 101010 → 0010 1010 → 2 A → 2A. Naturellement, cela fonctionne dans l'autre sens...

Hexadécimal	Décimal	Binaire		Hexadécimal	Décimal	Binaire
0	0	0000		8	8	1000
1	1	0001		9	9	1001
2	2	0010		A	10	1010
3	3	0011		B	11	1011
4	4	0100		C	12	1100
5	5	0101		D	13	1101
6	6	0110		E	14	1110
7	7	0111		F	15	1111

3. Codage des entiers relatifs

3.1. Le problème

Comment représenter les nombres négatifs avec seulement des 0 et des 1 ?

3.2. Signe et valeur absolue

Le bit de poids fort indique le signe (0 → le nombre est positif, 1 → il est négatif), les autres bits donnent la valeur absolue.

Problèmes :

- il existe alors deux représentations de zéro (+0 et -0), c'est gênant.
- l'addition ne fonctionne pas directement. En effet, sur 4 bits : $1_d = 0001_b$; $-1_d = 1001_b$; et $0001_b + 1001_b = 1010_b = -2_d$!

C'est pourquoi cette approche historique est très peu utilisée, en pratique.

3.3. Complément à 2

Le complément à 2 est la méthode utilisée par tous les processeurs modernes.

3.3.1. Principe sur n bits

- Les entiers positifs (de 0 à $2^{n-1} - 1$) sont représentés normalement.
- Pour un entier négatif $-x$, on code $2^n - x$.
- Conséquence : le bit de poids fort est 1 si et seulement si le nombre est négatif.

3.3.2. Méthode pratique pour obtenir le complément à 2 de $-x$

1. Écrire x en binaire sur n bits.
2. Inverser tous les bits (complément à 1 : les 1 deviennent des 0 et les 0 sont changés en 1).
3. Ajouter 1.

Exemple : coder -42_d (sur 8 bits) ▼

- 42_d s'écrit en binaire sur 8 bits : 00101010_b .
- Inversion des bits : 11010101_b
- Ajout de 1 : $11010110_b \leftarrow$ représentation de -42 .

Vérification : $42 + (-42) = 00101010 + 11010110 = 100000000$. En ne gardant que les 8 bits de poids faible : on obtient $00000000 = 0$ (on « oublie » le bit le plus à gauche). 

Remarque : complément à 1 ▼

On nomme ainsi la technique consistant à changer les 0 en 1 et les 1 en 0, car lorsqu'on additionne le nombre initial à celui résultant de cette transformation, on obtient un nombre composé *uniquement* de 1.

3.3.3. Plage de valeurs sur n bits

Format	Plage
Non signé (n bits)	0 à $2^n - 1$
Signé complément à 2 (n bits)	-2^{n-1} à $2^{n-1} - 1$

« Signé » signifie (nombre) « ayant un signe » 😊.

Concrètement, sur 8 bits :

- Pour un entier non signé, on peut coder les valeurs allant de 0 à 255.
- Pour un entier signé (pouvant être négatif), les valeurs iront de -128 à 127 .

Remarque : et en Python ? ▼

Python gère nativement les entiers de taille *arbitraire* (pas de dépassement : il n'oublie jamais, volontairement ou non, le bit de poids fort). En pratique, pour simuler le comportement sur n bits :

```
n = 8
x = -42
complement_a_2 = x % (2**n)    # 214 – représentation non signée de -42 sur 8 bits
bin(complement_a_2)            # '0b11010110'
```

Remarque : pour la culture : petit-boutisme et gros-boutisme ▼

Quand un entier occupe plusieurs octets en mémoire, deux conventions existent pour ordonner ces octets :

- **Gros-boutisme** (*big-endian*) : l'octet de poids fort est stocké en premier (adresse mémoire la plus basse). Utilisé par les protocoles réseau (TCP/IP).
- **Petit-boutisme** (*little-endian*) : l'octet de poids faible est stocké en premier. Utilisé par les processeurs x86/x64 (Intel, AMD) et ARM en configuration par défaut.

Exemple : considérons le nombre codé en hexadécimal par `0x1234` (sur 2 octets).

- En contexte gros-boutiste, il sera écrit / stocké par : `12 34`.
- En contexte petit-boutiste, il sera mémorisé par : `34 12`.

Cela n'a aucune incidence sur la programmation Python (qui fait abstraction de cela), mais c'est crucial lors d'échanges de données binaires entre machines d'architectures différentes.

```
import sys
sys.byteorder          # 'little' sur x86/x64
(0x1234).to_bytes(2, byteorder='big')    # b'\x12\x34'
(0x1234).to_bytes(2, byteorder='little')  # b'\x34\x12'
```

4. Codages complémentaires

4.1. Hexadécimal

L'hexadécimal n'est pas un codage à proprement parler : c'est une **notation compacte** du binaire. 4 bits $\rightarrow$ 1 chiffre hexadécimal. Elle est très utilisée pour afficher des adresses mémoire, des couleurs web (`#FF5733`), des hachages cryptographiques, etc.

4.2. BCD (Binary-Coded Decimal)

Remarque : pour la culture ▼

Le BCD code chaque chiffre décimal séparément sur 4 bits.

Exemple : 42 en BCD $\rightarrow$ 0100 0010 (4 codé 0100, 2 codé 0010).

Avantage : conversion décimal $\leftrightarrow$ BCD triviale, sans calcul. Utilisé dans les calculatrices, les afficheurs 7 segments, les systèmes embarqués traitant des montants financiers (évite les erreurs d'arrondi).

Inconvénient : moins compact que le binaire pur (6 combinaisons sur 16 sont inutilisées par quartet).

5. Représentation approchée des réels : les flottants

Les nombres réels ne peuvent pas tous être représentés exactement en binaire — la plupart ne sont qu'**approximés**. Python (comme la quasi-totalité des langages) utilise la norme **IEEE 754** sur 64 bits (*double précision*).

Conséquences pratiques :

```
0.1 + 0.2                # 0.30000000000000004  — pas 0.3 !
0.1 + 0.2 == 0.3          # False ← ne JAMAIS tester l'égalité de deux flottants

# À faire à la place :
abs(0.1 + 0.2 - 0.3) < 1e-9  # True — comparer avec une tolérance de 10-9
```

Pourquoi ? poser la division 1/10 en base 2 est une opération qui se poursuit à l'infini, exactement comme la division 1/3 ne se termine jamais en décimal. Seule une approximation tient en mémoire.

Quelques valeurs exactement représentables : 0, 5 (soit 2⁻¹), 0, 25 (ou 2⁻²), 0, 125, etc. — toutes les puissances *négatives* de 2. Naturellement, des nombres dont la partie décimale est composée d'une somme de puissances *négatives* de 2 sont parfaitement représentables en binaire.

6. Représentation du texte

6.1. Bref historique

Norme	Date	Taille	Caractères
ASCII	1963	7 bits	128 (lettres latines, chiffres, ponctuation)
ISO-8859-1 (<i>Latin-1</i>)	1987	8 bits	256 (ASCII + caractères ouest-européens)
Unicode	1991	variable	>140 000 caractères (tous les systèmes d'écriture)

Le problème d'ASCII : pas d'accents, pas de caractères non latins. Chaque région du monde a créé sa propre extension (ISO-8859-x, Windows-125x...), incompatibles entre elles. Unicode résout définitivement ce problème.

6.2. Unicode : points de code et encodages

Unicode définit un **point de code** (*code point*) pour chaque caractère : un entier unique, noté `U+XXXX` en hexadécimal.

Exemples :

Caractère	Point de code	Description
A	U+0041	Lettre latine majuscule A
é	U+00E9	Lettre e accent aigu
€	U+20AC	Signe euro
😊	U+1F600	Visage souriant

Un point de code n'est **pas** encore un codage : il faut choisir comment stocker cet entier en mémoire. C'est le rôle des **encodages**.

6.3. Encodages UTF-8 et UTF-32

UTF-32 : chaque caractère est stocké sur exactement **4 octets**. Simple, mais coûteux en espace (un texte en anglais utilise 4× plus de mémoire qu'en ASCII).

UTF-8 : encodage à **longueur variable** (1 à 4 octets selon le caractère). Rétrocompatible avec ASCII (les 128 premiers caractères tiennent sur 1 octet). C'est l'encodage dominant sur le web (>98% des pages en 2024).

Exemple : l'emoji 😊 (U+1F600)

Encodage	Représentation hexadécimale	Taille
UTF-32	00 01 F6 00	4 octets
UTF-8	F0 9F 98 80	4 octets
UTF-16	D8 3D DE 00 (paire surrogate)	4 octets

Pour cet emoji, les trois encodages coïncident en taille — car U+1F600 est hors du Plan Multilingue de Base. En revanche, pour A (U+0041) :

Encodage	Représentation	Taille
UTF-32	00 00 00 41	4 octets
UTF-8	41	1 octet

En Python :

```
emoji = '😊'
print(ord(emoji))           # 128512  – point de code (décimal)
print(hex(ord(emoji)))      # 0x1f600 – point de code (hexa)

emoji.encode('utf-8')       # b'\xf0\x9f\x98\x80' – 4 octets
emoji.encode('utf-32-be')   # b'\x00\x01\xf6\x00' – 4 octets

len(emoji.encode('utf-8'))  # 4
len('A'.encode('utf-8'))    # 1
len('A'.encode('utf-32-be')) # 4
```

i En pratique

En Python 3, les chaînes de caractères (`str`) sont nativement en Unicode. L'encodage n'entre en jeu que lors de la lecture/écriture de fichiers ou de communications réseau.

Il est prudent de chercher à déterminer l'encodage d'un fichier avant de l'ouvrir (en lecture comme en écriture). Il faut alors préciser cet encodage à l'ouverture d'un fichier, par exemple :

```
with open('fichier.txt', 'r', encoding='utf-8') as f:
    contenu = f.read()
```